

Welcome Offer Access to everything. Now 30% off. [Upgrade now](#)

# React `useState` Explained: Batching, Functional Updates, Render Phase & More



Devendra Dhote

Follow

4 min read · Jun 26, 2026



205

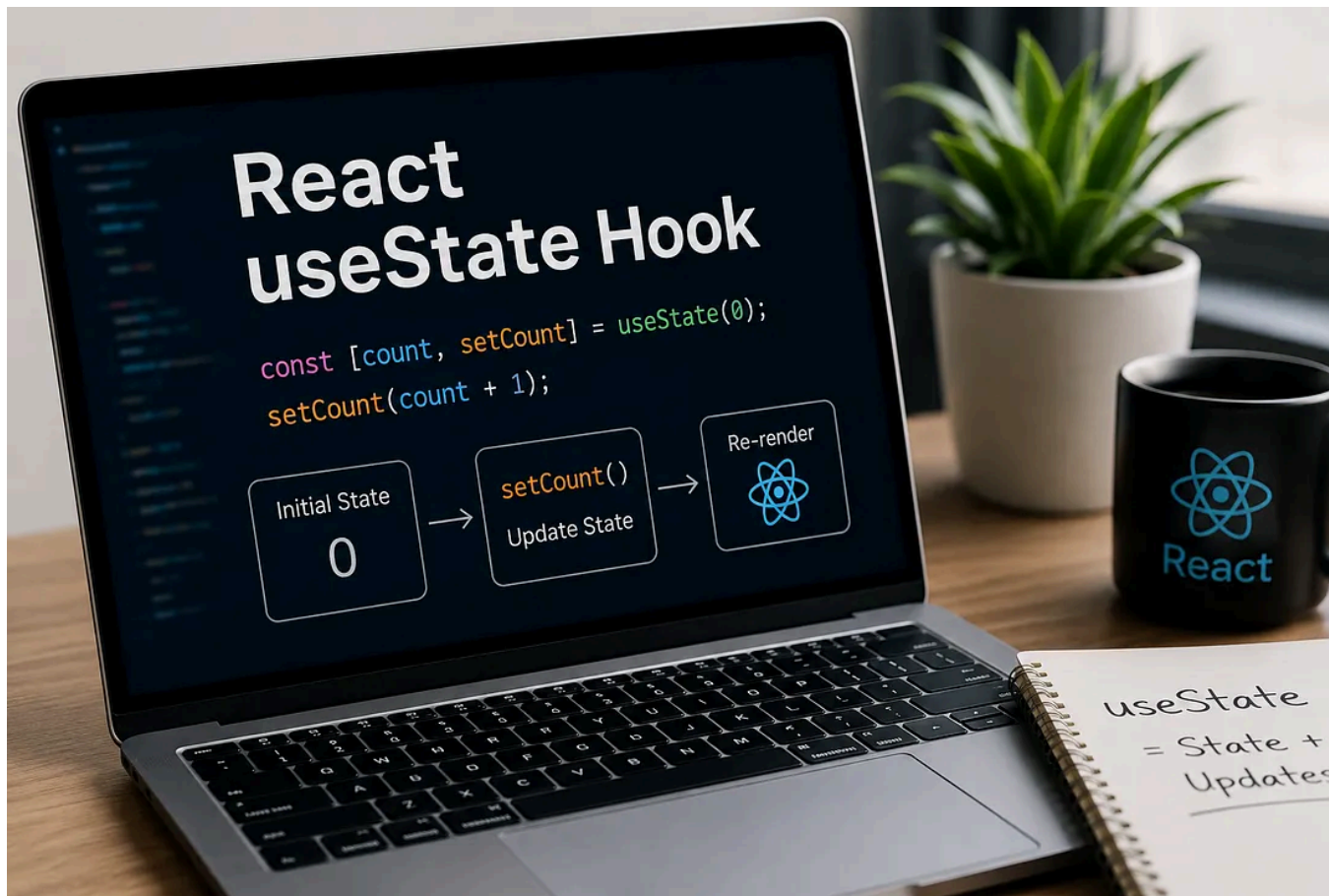


4



11





If you've been learning React, you've probably used the `useState` hook countless times.

```
const [count, setCount] = useState(0);
```

Most tutorials stop here and simply say, “*It updates the state and re-renders the component.*”

But what actually happens behind the scenes when we call `setCount()` ?

After digging deeper into React’s rendering process, I realized there’s much more happening than just changing a variable.

Let’s understand how `useState` really works.

. . .

## What is State?

State is React’s way of storing data that can change over time.

Whenever the state changes, React updates the UI to keep it in sync with the latest data.

```
const [count, setCount] = useState(0);
```

Here:

- `count` → Current state
- `setCount` → Function used to update the state
- `0` → Initial state

. . .

## What Happens When We Call `setState`?

Many beginners think this happens:

```
setCount(5);
```

State becomes `5` immediately.

But that's not how React works.

When we call `setCount()`, React doesn't update the state instantly.

Instead, it schedules the update.

The flow looks like this:

```
setCount()
```

↓

Update gets queued

↓

Render Phase

↓

Reconciliation

↓

Commit Phase

↓

UI gets updated

This scheduling is one of the reasons React remains fast and efficient.

. . .

## **Render Phase vs Commit Phase**

Understanding these two phases makes React much easier to reason about.

### **Render Phase**

During this phase React:

- Creates a new Virtual DOM
- Compares it with the previous one
- Finds what has changed
- Prepares the required updates

At this point, the real DOM hasn't changed yet.

. . .

### **Commit Phase**

Once React knows what changed, it enters the Commit Phase.

Here React:

- Updates the real DOM
- Updates the UI
- Runs effects like `useEffect`

This is the only phase where the browser actually sees any changes.

• • •

## What If We Update the State With the Same Value?

Consider this example:

```
const [count, setCount] = useState(0);
```

```
setCount(0);
```

The current state is already `0`.

The new state is also `0`.

React processes the update, but since nothing actually changed, it skips updating the UI.

No unnecessary DOM updates happen.

This optimization helps React avoid extra work.

. . .

## Automatic Batching

One of my favorite React optimizations is **Automatic Batching**.

Suppose we write:

```
setCount(1);  
setName("Dev");  
setAge(24);
```



Does React render three times?

No.

Instead, React groups these updates together.

Multiple Updates

↓

Batch Them

↓

Single Render

↓

Single Commit

This reduces unnecessary rendering and improves performance.

. . .

## The Famous Counter Example

Most beginners expect this code to increase the count by 3.

```
setCount(count + 1);  
setCount(count + 1);  
setCount(count + 1);
```

Assume the current value of `count` is `0`.



React sees:

```
setCount(1);  
setCount(1);  
setCount(1);
```

Since every update is trying to set the state to the same value, the final state becomes:

1

That's why the counter increases only once.

. . .

## The Magic of Functional Updates

Now look at this:

```
setCount(prev => prev + 1);  
setCount(prev => prev + 1);  
setCount(prev => prev + 1);
```

This time the result is:

3

Why?

Because `prev` is **not** always the original state.

It represents the **latest state available while React processes the update queue.**

The execution looks like this:

Current State

0

↓

First updater

`prev = 0`

↓

1

```
↓  
Second updater  
prev = 1  
↓  
2  
↓  
Third updater  
prev = 2  
↓  
3
```

This is exactly why React recommends using functional updates whenever the next state depends on the previous one.

. . .

## State Updates Are Scheduled

Another common mistake is expecting the state to change immediately.

```
setCount(10);
```

```
console.log(count);
```

Many developers expect:

10

But the output is still the previous value.

That's because `setState()` schedules an update instead of changing the value immediately.

React updates the state during the next render cycle.

• • •

## Stale Closures

Consider this example:

```
setTimeout(() => {  
  setCount(count + 1);  
}, 3000);
```

The callback remembers the value of `count` from the moment it was created.

If the state changes before the timeout finishes, the callback still uses the old value.

The safer approach is:

```
setCount(prev => prev + 1);
```

Since `prev` always receives the latest queued state, it avoids stale data problems.

• • •

## The Update Queue

Internally, React maintains an update queue.

Whenever we call:

```
setCount(1);  
setCount(2);  
setCount(3);
```

React doesn't execute them immediately.

Instead, it stores them in a queue.

```
setState()
```

↓

Update Queue

↓

React Processes Updates

↓



Render

↓

Commit

This queue is also the reason why batching and functional updates work so efficiently.

. . .

## Final Thoughts

The `useState` hook looks simple on the surface, but a lot happens behind the scenes.

Calling `setState()` doesn't immediately change the state.

Instead, React schedules the update, processes it during the Render Phase, batches multiple updates together whenever possible, and finally applies the changes during the Commit Phase.

Understanding concepts like batching, update queues, functional updates, and stale closures completely changes the way you think about React.

The next time you write:

```
setCount(prev => prev + 1);
```

you'll know that React is doing much more than simply incrementing a number.

React ✓

JavaScript ✓

Coding +

Usestate +

Programming ✓



**Written by Devendra Dhote**

284 followers · 8 following

MERN Stack Developer & Instructor. Writing about JavaScript, React, Next.js, backend development, and the engineering concepts behind modern web applications.

Follow